

INTERNSHIP REPORT

ON

PROGRAMMING C LANGUAGE

BY

KANIKA

UNIVERSITY ROLL NO. 2024242307

DEPARTMENT OF COMMERCE

GOVERNMENT COLLEGE AHARWALA (BILASPUR)

(KURUKSHETRA UNIVERSITY, KURUKSHETRA)

YEAR OF SUBMISSION - 2026

Acknowledgment

I would like to express my deepest gratitude to all those who have guided, supported and encouraged me throughout the course of my internship.

First and foremost, I would like to extend my sincere thanks to Dr. Sarvjeet Kaur, Principal Government College, Aharwala (Bilaspur), for her inspiring leadership, encouragement and providing me this opportunity, resources and a platform to enhance my knowledge and skill,

I would like to extend my sincere thanks to Mr. Rahul Saini, Teacher of Authorized Training Partner of: M/s Global Infosys, and C Language Internship Mentor, for his constant guidance, motivation and valuable insights. I am also grateful to Ms. Anita Mam, Internship Supervisor, and Ms. Chanchal Rani, Internship Incharge, for their continuous support and encouragement. My heartfelt thanks Ms. Aarti Arora, Internship Coordinator, for her assistance and cooperation during the internship period.

Finally, extend a special thanks to my father, Mr. Satish Kumar for always being my strongest support system and providing me with the necessary resources, care, and encouragement to achieve my goals.

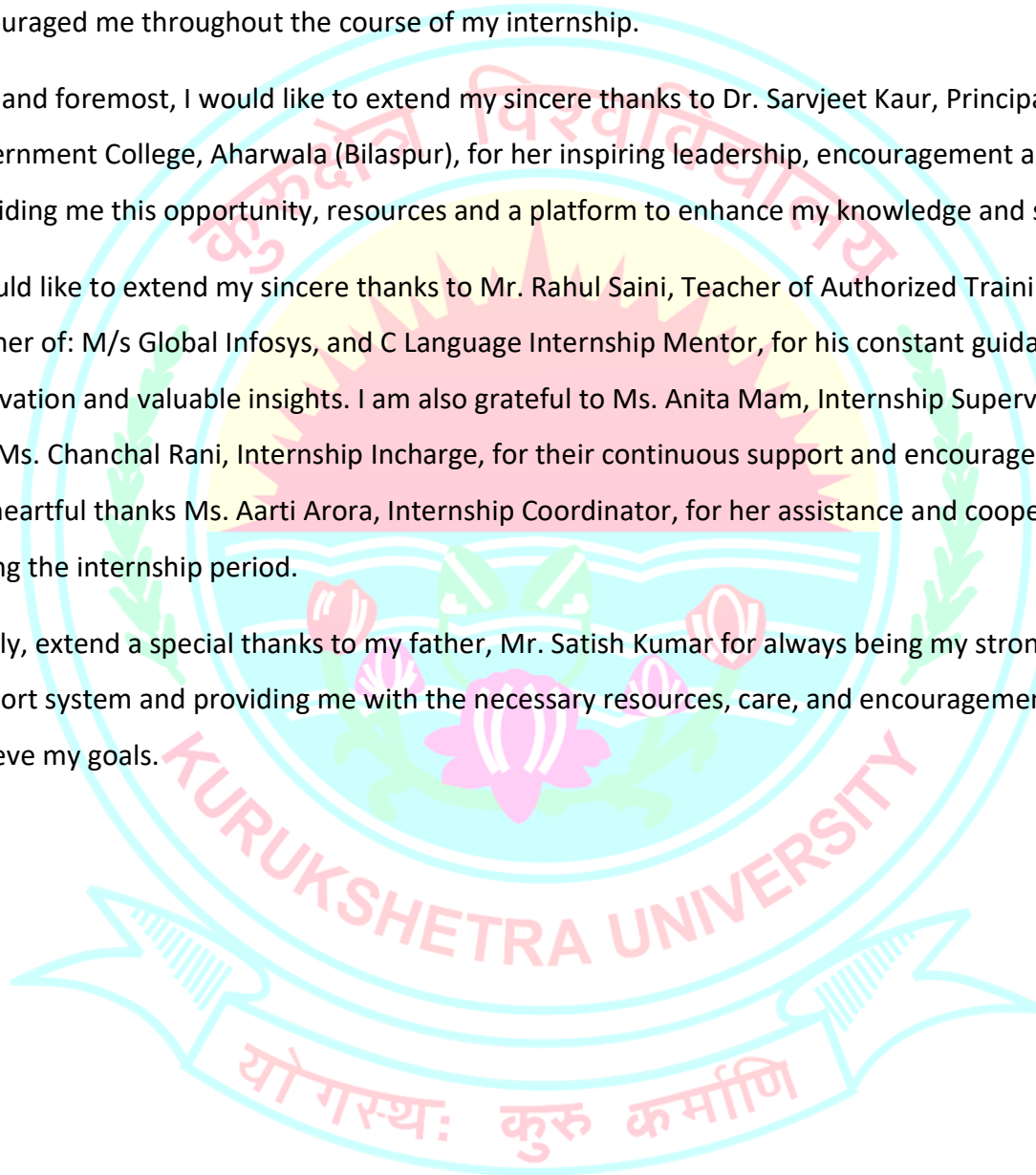


TABLE OF CONTENTS

- I. Certificate
- II. Acknowledgments
- III. About the organization
- IV. Objectives of Internship
- V. Details of Work done
- VI. Learning Experiences
- VII. Learning Outcomes
- VIII. Conclusion / Summery
- IX. Attachments



Introduction

A programming language is a way for people to give instructions to a computer. It uses special words and rules to create programs that tell the computer what to do. These instructions can make the computer perform tasks like solving problems, playing games, or running apps. Examples of programming languages are Python, Java, and C++. There are Three Types of Programming languages:

- High-Level Language
- Medium Level Language
- Low-Level Language

C is a programming language developed at AT&T's Bell Laboratories of USA in 1972. It was designed and written by a man named Dennis Ritchie. It was mainly created as a system programming language to write the UNIX operating system.

ABOUT THE ORGANIZATION

Global Infosys, Jagdhri is a computer education and IT training institute dedicated to providing technical education and practical knowledge to students. The organization focuses on developing programming, software development, and computer application skills through classroom instruction and hands-on laboratory practice. It aims to bridge the gap between academic learning and industry requirements by offering practical training and internship opportunities.

The institute provides training in various programming languages and software technologies, enabling students to build strong technical foundations. During the internship, students gain practical experience by writing programs, solving programming problems, and developing small software applications under the guidance of experienced instructors.

Global Infosys provides training in various computer-related subjects, such as:

- C Programming
- C++
- Java
- Python
- HTML, CSS, and JavaScript
- MS Office
- Internet Applications
- Database Management
- Software Development Basics

Global Infosys, Jagdhri provided practical training in the C programming language. The training covered fundamental concepts such as variables, data types, operators, decision-making statements, loops, functions, arrays, strings, pointers, structures, unions, and file handling. Students practiced writing, compiling, and testing, which helped improve their logical thinking and programming skills.

OBJECTIVES OF INTERNSHIP

- To gain practical knowledge of C Language.
- To understand the fundamentals of the C programming language and its applications.
- To develop logical thinking and problem-solving skills through programming.
- To learn the concepts of variables, data types, operators, decision making statements and looping constructs for program control.
- To understand decision-making statements and looping constructs for program control.
- To implement user-defined functions for modular programming.
- To learn the concepts of arrays, strings, pointers, structures, and unions.
- To understand the importance of structured programming in developing efficient and maintainable software

DETAIL OF WORK

Week 1- About the C language (Compiler, Variables and data types)

C is a programming language developed at AT&T's Bell Laboratories of USA in 1972. It was designed and written by a man named Dennis Ritchie. It was mainly created as a system programming language to write the UNIX operating system. C is often called the "mother of programming languages" because many modern programming languages, such as C++, Java, and C#, are influenced by it.

Compiler:

A compiler is a software program that converts the entire source code written in the C programming language into machine code (object code) before the program is executed. This machine code is then run by the computer.

When a programmer writes a C program, the computer cannot understand it directly because it only understands machine language (binary code). A compiler translates the C program into machine code, checks for syntax errors, and creates an executable file that can be run on the computer.

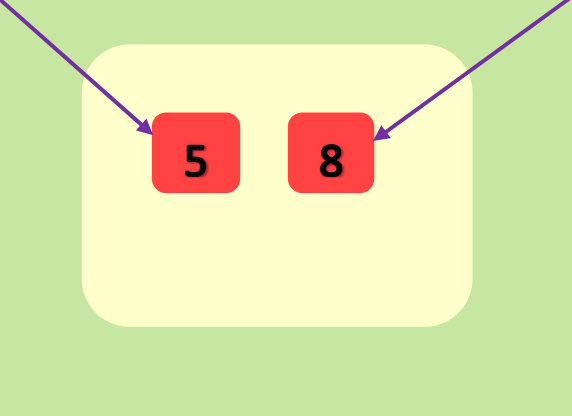
Variables:

A variable is a named memory location used to store data in a program. It can hold different types of values like integer, float, or character. The value of a variable can be changed during program execution. Variables make it easy to read, write, and manipulate data in C.

Variable Naming Rules:

- Variable name is any combination of alphabets, digits and underscore.
- No other symbol is allowed. `int x=5;`
- Variable name cannot start with digit. `int y=8;`

✓	✗	
x	9x	
x4	9_x	
_x	x-3	
x_1	x.b	
x	x\$	



The diagram illustrates a memory structure labeled 'RAM' as a green rectangle. Inside, a yellow rounded rectangle represents memory space. Two red boxes are shown: the first contains the number '5' and is labeled 'X Variable' with a purple arrow; the second contains the number '8' and is labeled 'Y Variable' with a purple arrow.

Data Types:

Why we need Data Types:

- Size required to store data.
- Internal binary representation data.
- Kind of operation

Data Type

Primitive

Non Primitive

Data Type	Size	Example
int	4 bytes	4, 54, -4
float	4 bytes	4.8, 19.5
double	8 bytes	453.24243
char	1 bytes	'A', '6', '\$'
void	-	-

array	int arr[5]={11,12,13,14,15};
string	char str[5]={ "Rahul" };
structure	{ int roll }
union	union Data { int i; float f; };
pointer	int *ptr;
enum	enum Days {SUN, MON, TUE};

Week 2: Operators

They are used to perform certain operations on the operands. For example: Arithmetic, Comparison, Logical, Bitwise, etc.

The operators can be divided into three categories based on the number of operands which they are to. In this case, the operators can be categorized as follow:

- Union operators
- Binary operators
- Requires three operators

The operators can be categorized on the basis of nature of operation they perform. In this case, the operators can be classified as:

Arithmetic operators:

+	-	*	/	%
4+5=9	5-3=2	5*2=10	10/2=5	9%2=1

Assignment operators: `int x=10;`

=	+=	-=	*=	/=	%=
x = 10	x += 2 12	x -= 2 8	x *= 2 20	x /= 2 5	x %= 3 1

Comparison operators:

==	!=	>	<	>=	<=
5 == 5 1	5 != 5 1	5 > 5 0	5 < 5 0	5 >= 5 1	5 <= 5 1

Logical operators:

&& (and)	(or)	! (not)
0 && 0 0	0 0 0	! 0 1
0 && 1 0	0 1 1	! 1 0
1 && 0 0	1 0 1	
1 && 1 1	1 1 1	

Bitwise operators: **Note:** Work with Binary Numbers!

& (and)	(or)	^ (xor)	~ (not)
0 & 0 = 0	0 0 = 0	0 ^ 0 = 0	~ 0 = 0
0 & 1 = 0	0 1 = 1	0 ^ 1 = 1	~ 1 = 1
1 & 0 = 0	1 0 = 1	1 ^ 0 = 1	
1 & 1 = 1	1 1 = 1	1 ^ 1 = 0	

Week 3: Control Instructions

1. Decision Control Instruction
2. Switch Case Control Statements
3. Iterative Control Instruction

1. Decision control Instructions:

A. If Else

B. Else If

A. If Else:

```
# include <stdio.h>

int main(){
    int a=1;
    if(a>0){
        printf("positive");
    }else{
        printf("negative");
    }
}
```

Output

positive

B. Else If:

Syntax

```
if (condition){  
    code;  
    code;  
}else{ code;  
}
```

Output

A+

```
# include <stdio.h>  
  
int main(){  
    int marks=450;  
    if(marks<=500 && marks>400){  
        printf("A+");  
    }else if(marks<=400 && marks>300){  
        printf("A");  
    }else if(marks<=300 && marks>200){  
        printf("B");  
    }else if(marks<=200 && marks>100){  
        printf("C");  
    }else if(marks<=100 && marks>0){  
        printf("Fail");  
    }else{  
        printf("Please Provide Suitable Value");  
    }  
}
```

2. Switch Case Control Statements:

Syntax

```
if (condition){  
    code;  
}else if(condition){  
    code;  
}else{  
    code;  
}
```

Output

Good

```
char grade = 'B';  
switch (grade)  
{  
    case 'A':  
        printf("Excellent");  
        break;  
    case 'B':  
        printf("Good");  
        break;  
    case 'C':  
        printf("Average");  
        break;  
    default:  
        printf("Invalid Grade");  
}
```

Week:4

3. Iterative Control Instruction:

- A. while
- B. do while
- C. for

A. While:

Syntax

```
variable;  
while(condition){  
code;  
increment;  
}
```

```
# include <stdio.h>  
  
int main(){  
    int x=1;  
    while(x<=5){  
        printf("%d\n",x);  
        x++;  
    }  
}
```

Output

```
1  
2  
3  
4  
5
```


B. Do while:

Syntax

variable;

do{

code;

increment;

}while(condition);

Output

1
2
3
4
5

```
#include <stdio.h>

int main(){
    int x=1;
    do{
        (x<=5);
        printf("%d\n",x);
        x++;
    }while(x<=5);
}
```

C. For:

Output

1
2
3
4
5

```
#include <stdio.h>

int main(){
    int x=1;
    for( int x=1 ; x <=5 ; x ++){
        printf("%d\n",x );
    }
}
```

Syntax

{for(variable; condition; increment)

code;

}

Functions: A function is a block of code that does a specific job. It runs only when it is called. It can take input and give output. It helps make the program simple and easy to understand. Using functions we can enable code reusability.

The return statement is used to exit a function and go back to the place from where it was called. return statement can contain an expression that gets evaluated and the value is returned.

Syntax

```
return_type function_name(parameters){  
    code;  
    [return[value]];  
}
```

Example:

Output

1

```
#include <stdio.h>  
int isPrime (int x){  
    for(int i=2;i<x;i++){  
        if(x%i==0){  
            return 0;  
        }  
    }  
    return 1;  
}  
int main() {  
    printf("%d\n",isPrime(13));  
}
```

Week 5: Array, Structure, Union, and Pointer

Array

An array is a collection of similar data types stored in contiguous memory locations. It allows you to store and access multiple values using a single variable name with an index. Basically it can store multiple variables in a single array.

Syntax

`data_type array_Name [size]`

Output

1
2
3
4
5

```
#include <stdio.h>
int main() {
    int arr[5] = {1, 2, 3, 4, 5};
    for (int i = 0; i < 5; i++){
        printf("%d\n", arr[i]);
    }
}
```

Structure

A structure is a non-primitive data type in C that lets you combine variables of different data types under one name. It is useful when you want to group related information together. Each variable in the structure is known as a member of the structure. You can create a structure by using the struct keyword and declare each of its members inside curly braces:

Syntax

```
struct struct_name
{
    data_type element 1;
    data_type element 2;
}
```

Output

Roll: 101

Name: Rahul

Marks: 85.5

```
#include <stdio.h>
struct Student{
    int roll;
    char name[20];
    float marks;
};

int main(){
    struct Student s1 = {101, "Rahul", 85.5};
    printf("Roll: %d\n", s1.roll);
    printf("Name: %s\n", s1.name);
    printf("Marks: %.2f\n", s1.marks);
}
```

Union

A union is similar to a struct in that it can store members of different data types.

However, there are some differences:

- In a struct, each member has its own memory.
- In a union, all members share the same memory, which means you can only use one of the values at a time.

Syntax

```
union union_name
{
    data_type element 1;
    data_type element 2;
}
```

Output

```
100
150.0
```

```
#include <stdio.h>
union A{
    int a;
    float b;
};
int main(){
    union A A1;
    A1.a=100;
    A1.b=150.0;
    printf("%d\n",A1.a);
    printf("%d\n",A1.b);
}
```

Pointer

A pointer is a variable that stores the memory address of another variable as its value. A pointer variable points to a data type (like int) of the same type, and is created with the * operator. The address of the variable you are working with is assigned to the pointer.

Syntax

`data_type*variable_name`

Output

9

0

12

```
#include <stdio.h>

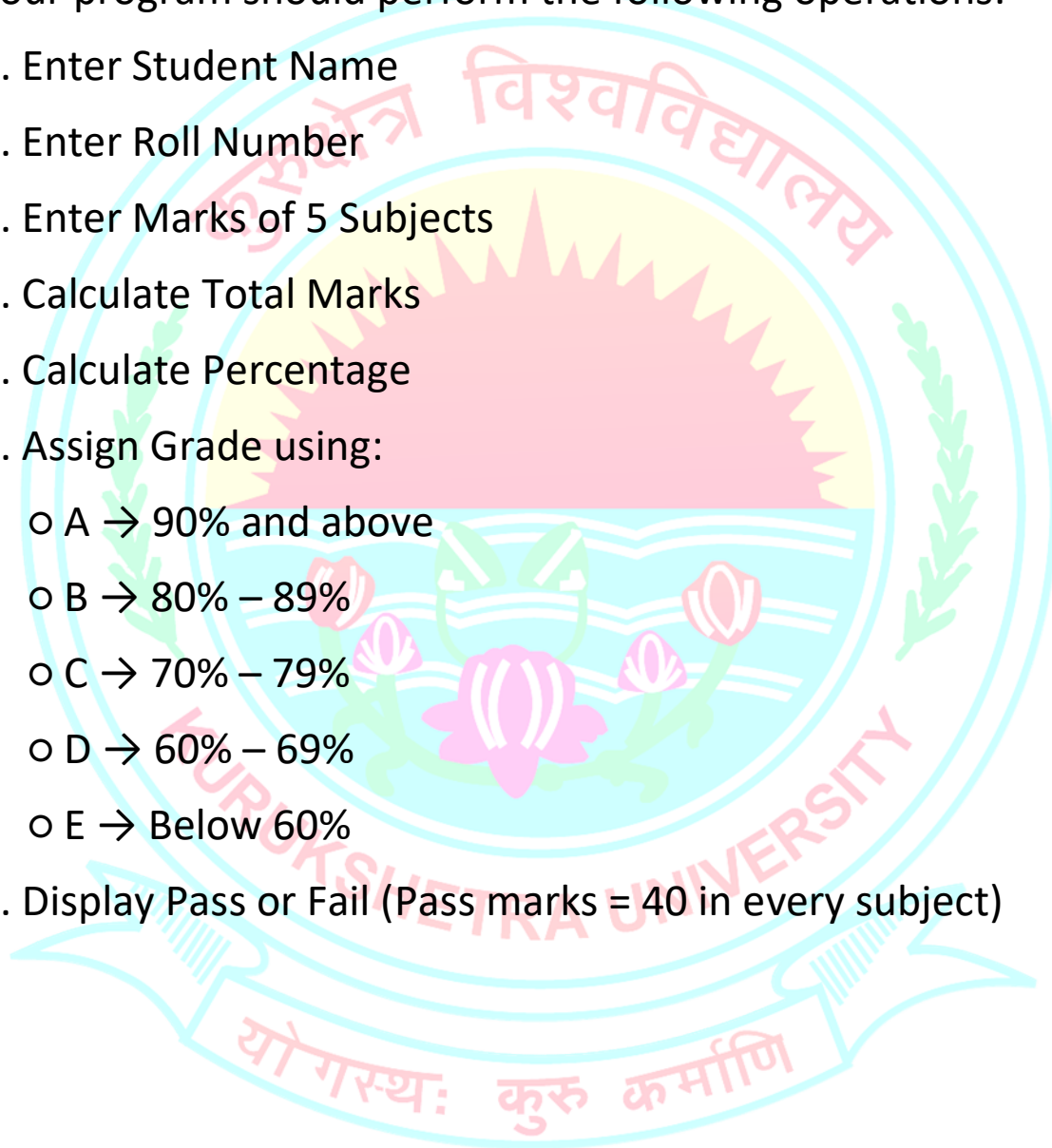
int main(){
    int x=9;
    int y=12;
    int *p;
    printf("%d\n",x);
    printf("%d\n",p);
    printf("%d\n",y);
}
```


Week 6: Project work: Student Report

Create a menu-driven program that allows the user to enter a student's details and display the result.

Your program should perform the following operations:

1. Enter Student Name
2. Enter Roll Number
3. Enter Marks of 5 Subjects
4. Calculate Total Marks
5. Calculate Percentage
6. Assign Grade using:
 - A → 90% and above
 - B → 80% – 89%
 - C → 70% – 79%
 - D → 60% – 69%
 - E → Below 60%
7. Display Pass or Fail (Pass marks = 40 in every subject)



```

#include <stdio.h>

int main() {
    printf("*****STUDENT RESULT*****");
    char name[50];
    int rollno;
    int hindi, english, math, science, computer, total, marks;
    float percentage;
    char grade[25];
    int pass=1;
    //name
    printf("\nEnter Student Name: ");
    scanf("%[^\n]", &name);
    //rollno
    printf("Enter Student Rollno: ");
    scanf("%d", &rollno);
    //marks
    printf("\nEnter marks of Hindi: ");
    scanf("%d", &hindi);
    printf("Enter marks of English: ");
    scanf("%d", &english);
    printf("Enter marks of Math: ");
    scanf("%d", &math);
    printf("Enter marks of Science: ");
    scanf("%d", &science);
    printf("Enter marks of Computer: ");
    scanf("%d", &computer);
    total=hindi+english+math+science+computer;
    percentage=total/5;
    //output
    printf("\n\n-----\nStudent Name: %s\n", name);
    printf("Student Roll no: %d", rollno);
    printf("\nTotal marks: %d", total);
    printf("\nPercentage: %d", percentage);
    //grade
    if (percentage >= 90) {
        printf("\nGrade: A");
    } else if (percentage >= 80) {
        printf("\nGrade: B");
    } else if (percentage >= 70) {
        printf("\nGrade: C");
    } else if (percentage >= 60) {
        printf("\nGrade: D");
    } else {
        printf("\nGrade: E");
    }
    //result
    if (pass) {
        printf("\nResult: Pass");
    } else {
        printf("\nResult: Fail");
    }
    printf("\n-----");

    return 0;
}

```

Output

*****STUDENT RESULT*****

Enter Student Name: Kanika

Enter Student Rollno: 101

Enter marks of Hindi: 85

Enter marks of English: 95

Enter marks of Math: 86

Enter marks of Science: 96

Enter marks of Computer: 89

Student Name: Kanika

Student Roll no: 101

Total marks: 451

Percentage: 4202792

Grade: A

Result: Pass

Learning Experience

- **Learned the basics of C programming**

You mastered fundamental syntax, keywords, and how a computer executes code.

- **Understood variables, loops, and functions**

You can store data, repeat actions efficiently, and break code into reusable blocks.

- **Learned how to use arrays and strings**

You can manage lists of numbers and process text data.

- **Gained knowledge of pointers and file handling**

You learned to manage computer memory directly and save data permanently to hard drives. [[1](#), [2](#), [3](#), [4](#), [5](#)]

- **Practiced writing simple C programs**

You built muscle memory by turning abstract concepts into working software.

- **Learned to find and fix errors in programs**

You developed debugging skills to isolate syntax errors and runtime bugs.

- **Worked on small programming projects**

You combined multiple small concepts to build larger, functional applications.

- **Learned to write clean and organized code**

You adopted industry standards like proper indentation, naming conventions, and comments. [[1](#), [2](#), [3](#), [4](#)]

- **Understood how software is developed**
You learned the software lifecycle, from writing code to testing and deployment.
- **Completed tasks within the given time**
You developed time management skills and learned to meet project deadlines.
- **Learned from mentors and their guidance**
You took constructive feedback to accelerate your learning curve.
- **Improved communication and teamwork skills**
You learned to explain your technical ideas and collaborate with other developers.
- **Gained confidence in programming**
You overcame the initial steep learning curve and trust your ability to build software. [[1](#), [2](#), [3](#), [4](#)]



Learning Outcome

- **Improved logical thinking**
Coding forced you to think step-by-step like a computer. You learned to organize your thoughts in a strict, structured sequence. [[1](#), [2](#), [3](#)]
- **Developed problem-solving skills**
You learned how to take a large, complex problem and break it down into smaller, manageable pieces of code. [[1](#), [2](#)]
- **Learned to find and fix errors**
Also known as debugging. You learned to read error messages, trace variables, and systematically hunt down bugs in your logic. [[1](#), [2](#), [3](#), [4](#)]
- **Learned to write simple C programs**
You progressed from reading theory to actively building working tools, such as calculators or basic games.
- **Improved coding skills**
Your code became cleaner, faster, and more efficient through hands-on practice and repetition. [[1](#), [2](#), [3](#), [4](#), [5](#)]
- **Gained practical programming experience**
You shifted from passive learning to active creation. You now understand the real-world workflow of writing, compiling, and running code. [[1](#), [2](#)]
- **Improved confidence in coding**
Overcoming syntax errors and broken logic proved to you that you can handle technical challenges, making future languages easier to learn. [[1](#), [2](#)]

Conclusion

The C Language internship was a valuable and enriching learning experience that helped me improve both my technical and professional skills. During the internship, I gained practical knowledge of C programming concepts such as variables, loops, functions, arrays, pointers, structures, and file handling. I also learned how to write, test, debug, and optimize programs while applying theoretical concepts to real programming tasks. The internship enhanced my logical thinking, problem-solving abilities, and coding confidence. In addition, it helped me develop important skills such as teamwork, communication, time management, and discipline in completing assigned tasks. The guidance provided by mentors and the hands-on experience gained throughout the internship strengthened my understanding of programming and prepared me for future academic projects and career opportunities in software development. Overall, the internship successfully bridged the gap between classroom learning and practical application, making it a meaningful step in my professional growth.

